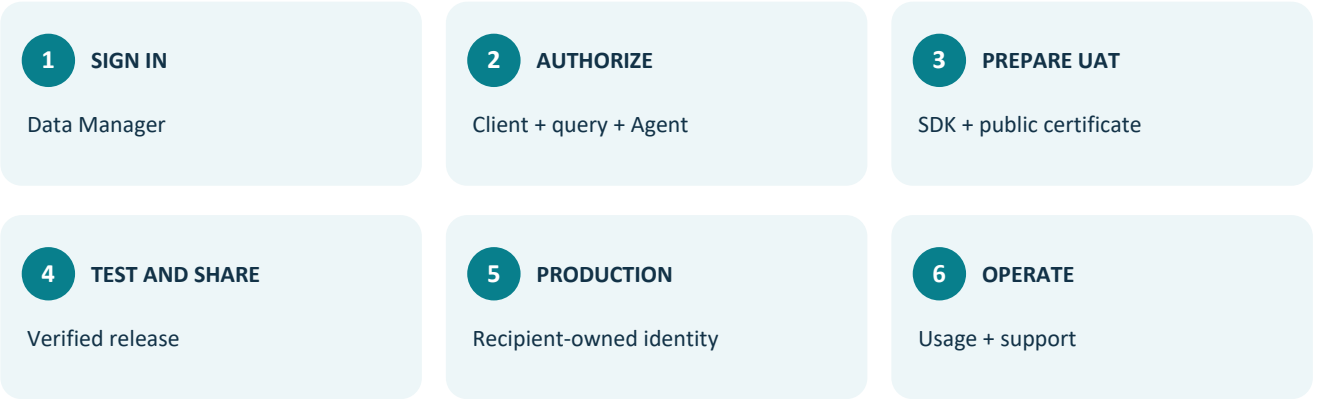


Data sharing through API

A first-time Data Manager's guide

From signing in to delivering a working UAT kit and preparing a Production integration. Follow the numbered steps, check the result on each screen, and use the handoff checklists before sharing a release.



You will prepare	You will hand over
An API client with approved queries, Agent scope and access rules.	A client-specific SDK ZIP, recipient README and SHA-256 checksum.
UAT identity registration and a verified test call.	A reviewed email with an approved secure transfer link.
Production authorization and recipient public-certificate registration.	A recorded acceptance result and support reference.

Checkpoint

All screenshots use demonstration data. TrainingPartnerAPI, TRAIN-01 and 192.0.2.10 are examples, not working credentials or deployment instructions. No live business records were created for this manual.

Edition: current local DOORS implementation reviewed on 18 September 2026. This is an operating guide, not a security certification. Your approved portal address, client, certificate, gateway and business permissions take precedence over examples.

Your route through this manual

Stage	Pages	What you achieve
Prepare and sign in	3-4	Know the terms, collect approvals and enter Data Manager mode.
Register and authorize	5-10	Create a client; map the query and Agent; check IP and protocol.
Choose the kit	11-12	Select UAT or Production; identify the five downloaded files.
Complete UAT	13-18	Set up, register the certificate, test, build, verify and share.
Prepare Production	19-21	Build a separate release; recipient installs its own identity; handoff.
Test and operate	22-25	Use browser testing, subscription mapping, hours and troubleshooting.
Close the handoff	26-27	Sign-off checklist, source references and current guide limitations.

Who does what

Data Manager: creates the API client, authorizes query and Agent scope, maintains access policies, registers public certificates, coordinates the release and records acceptance.

Build custodian / technical support: runs the generated scripts on an approved Windows workstation, handles JDK and certificate trust, verifies the release files and supports failures. A beginner can follow the steps with this person present.

Recipient integration team: verifies the download, supplies runtime parameters and integrates the Java SDK. In Production it owns and retains the private PKI identity.

Checkpoint

Complete the UAT route first. The Production download button changes the SDK profile; it does not create a new API key, move a client to another environment, or approve Production access.

Before your first login

Term	In plain language
API client / API entry	The receiving application registered in DOORS. Its API key identifies machine requests.
Query / SQL template	An approved report definition. The recipient sends parameter values, not a new SQL statement.
Agent	The approved GePNIC node from which DOORS obtains data.
API user	A person/account using the subscriber portal. It is different from the machine API client.
SDK	Generated Java source, compiled client and supporting files used to call DOORS.
Public certificate / private P12	The public certificate is registered in DOORS. The P12 contains the private identity used by the SDK.
UAT / Production	Controlled acceptance testing / approved live operation.
SHA-256 checksum	A file fingerprint used to confirm that the received ZIP matches the released ZIP.

Collect these details before starting

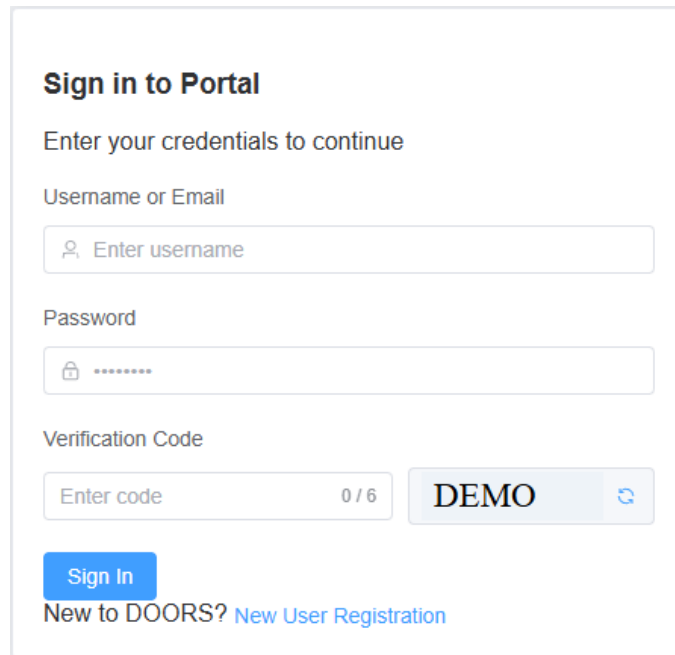
Your DataManager account and approved admin portal address; recipient organization and technical contact; application name; approved Query Name and its parameter list; approved Agent ID; recipient server's outbound IP address; UAT scope and validity; support/approval reference; and an approved protected transfer channel.

For SDK preparation, arrange Windows PowerShell and JDK 21 or later with **java**, **javac** and **keytool** available. For Production, ask the recipient's PKI team to prepare its organization-owned identity and approved gateway trust configuration.

Checkpoint

The screenshots use DailyTenderSummary with parameter p_summary_date. Obtain the actual query name, required parameters and expected result from the query owner. Do not create an unapproved report just to reproduce this example.

Sign in as Data Manager

The image shows a web form titled "Sign in to Portal". Below the title is the instruction "Enter your credentials to continue". There are three input fields: "Username or Email" with a placeholder "Enter username", "Password" with a placeholder "Password" and a lock icon, and "Verification Code" with a placeholder "Enter code" and a character count "0 / 6". To the right of the verification code field is a button labeled "DEMO" with a refresh icon. Below the input fields is a blue "Sign In" button. At the bottom, there is a link "New to DOORS? [New User Registration](#)".

Sign in to Portal

Enter your credentials to continue

Username or Email

Enter username

Password

Password

Verification Code

Enter code 0 / 6

DEMO

Sign In

New to DOORS? [New User Registration](#)

Figure 1. The sign-in screen. The verification image is a demonstration.

1. Open the approved **admin portal**. In this local installation the example is <https://doorsadmin.doors.test/doors/login>. Use your administrator-provided address in another environment.
2. Enter your **Username or Email** and **Password**. Enter the current **Verification Code** shown beside the field; refresh it if unreadable.
3. Select **Sign In**. If prompted, complete authenticator enrollment or enter the six-digit code and choose **Verify & continue**. Complete a required first-login password change if shown.
4. Check that the dashboard shows **Data Manager MODE**. Expand **Governance Hub** in the left menu.

Checkpoint

Expected result: the Data Manager dashboard and governance menu are visible. If the role is wrong, sign out and use the correct account. Same-host tabs share a login. If a certificate warning appears, stop and ask support to fix trust; do not bypass it.

Find the API Client Registry

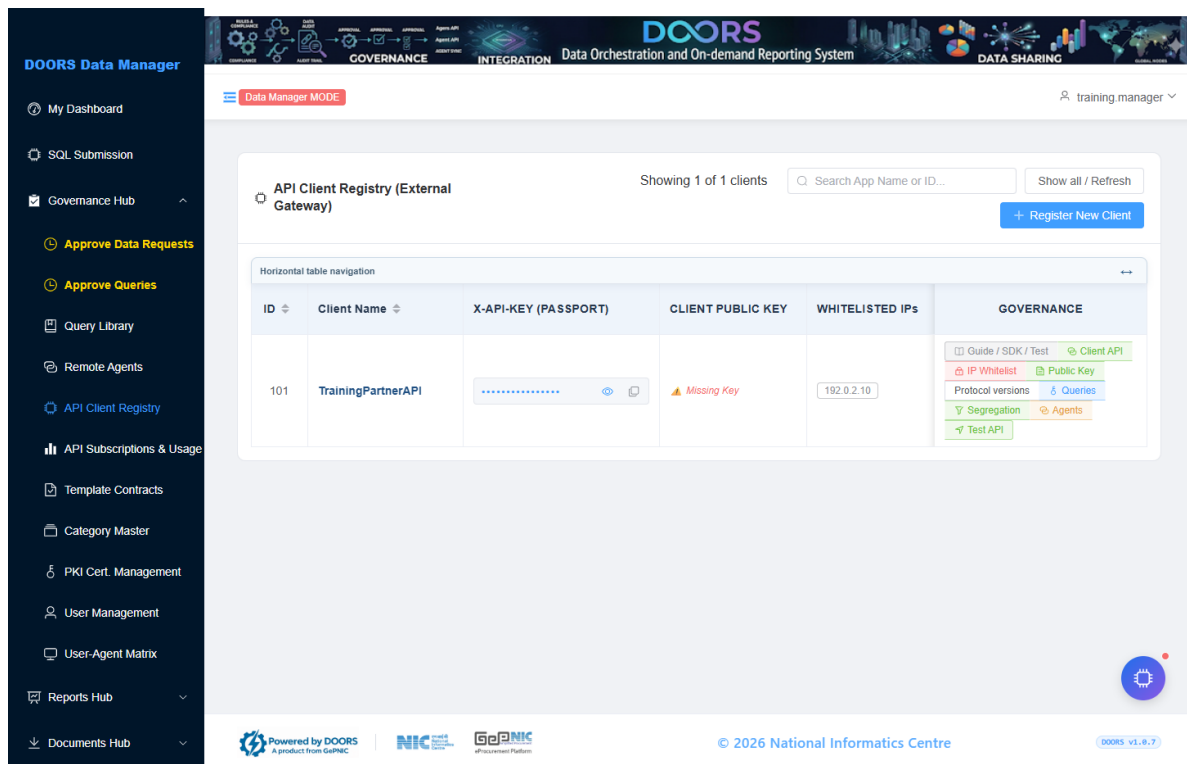


Figure 2. Registry with one fictional application; credentials remain masked.

1. Open **Governance Hub** → **API Client Registry**. The page heading is **API Client Registry (External Gateway)**.
2. Use **Search App Name or ID...** to check whether the application already exists. Use **Show all / Refresh** if you cannot find it.
3. Locate the application row and its **GOVERNANCE** buttons on the right. The horizontal scrollbar reveals columns hidden on a narrow screen.
4. For a new application choose **Register New Client**. For an existing one, confirm its client ID and owner before changing anything.

Use the right button

Queries selects reports. **Agents** selects source nodes. **IP Whitelist** limits calling IP addresses. **Public Key** registers the SDK identity. **Guide / SDK / Test**, **Client API** and **Protocol versions** open the version-policy window.

Checkpoint

Treat the API key like a password. Do not reveal or paste it into a training screenshot, email or ticket. The generated setup script already carries the selected client's key.

Create the API entry

Register New Client

Application Name

TrainingPartnerAPI

Client Public Key (Base64 .PEM Content)

Paste Base64 Key block string

Whitelisted IPs

192.0.2.10

Leave empty to allow all (not recommended)

Cancel

Confirm Registration

Figure 3. Registration form with example name and documentation-only IP.

- 1. Enter the approved **Application Name**, for example TrainingPartnerAPI. Use a name that identifies the receiving application and environment.
- 2. Leave **Client Public Key** blank during registration. In the current implementation, register the certificate separately using the row's **Public Key** action (page 15); do not assume this registration-form field saves it.
- 3. Enter the approved outbound IPv4/IPv6 addresses in **Whitelisted IPs**. Ask the recipient's network team for the address seen by DOORS, including any NAT or proxy.
- 4. Select **Confirm Registration**. Wait for the success message and locate the new row. Record the client ID, application name and approval reference.

Checkpoint

Expected result: a new registry row is visible. Registration alone does not authorize reports or Agents. The displayed 192.0.2.10 is a documentation address; replace it. Empty IP restrictions allow any source IP and are not the normal release setting.

Map the approved query

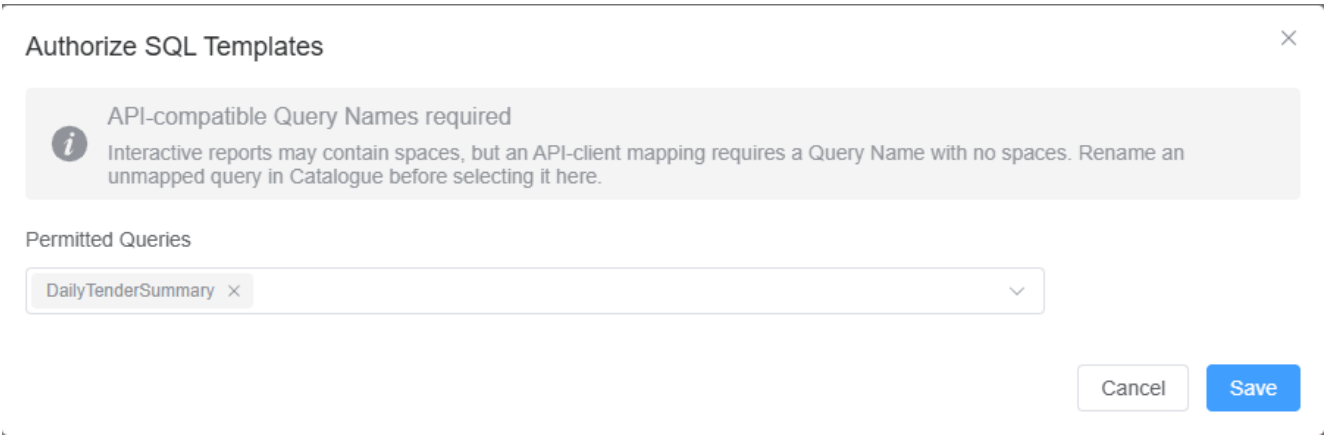


Figure 4. Queries opens Authorize SQL Templates.

- 1. On the correct client row, select **Queries**. Open **Permitted Queries** and search for the approved Query Name.
- 2. Select the authorized query or queries. Review the whole selection: removing an existing item changes that client's permitted reports.
- 3. Choose **Save**. Refresh the registry and check the **AUTHORIZED QUERIES** column.
- 4. Record each query's exact name and required parameters. For the example report, a runtime input is **p_summary_date=2026-09-01**. In the SQL template the placeholder may be **:p_summary_date**; the command-line argument omits the colon.

If the query is missing

Check **Governance Hub → Query Library → Approved**. A pending or rejected query is not available for API mapping. An API Query Name must be 3-100 characters, start with a letter, and use only letters, digits, underscores or hyphens. Use **Catalogue** metadata to correct an unmapped name before mapping, following your change process.

If a new report is needed, use **SQL Submission** with the query owner: provide its Unique Name, category, target Agent, SQL and justification, validate it, and select **Submit for Review**. The authorized reviewer uses **Approve Queries** and **Approve & Authorize**. Return here only after approval.

Checkpoint

Do not send arbitrary SQL in the API call. Use the approved query and its named inputs. Do not change SQL security filters or replace parameters with concatenated SQL to make a test pass.

Map the source Agent

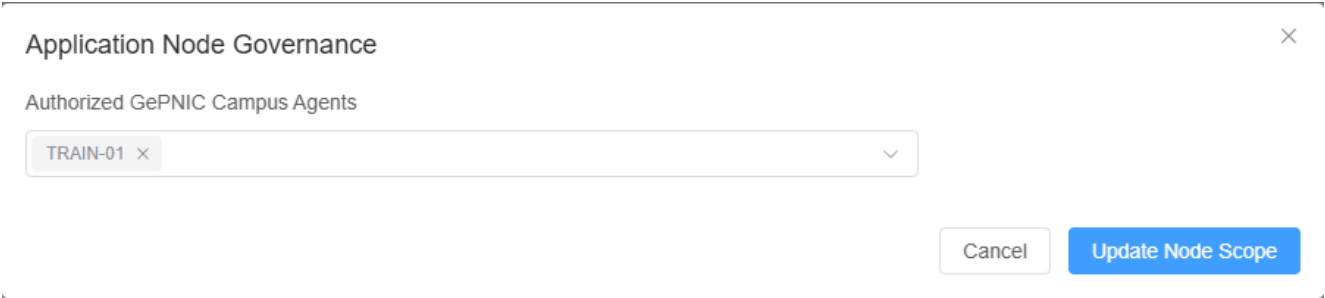


Figure 5. Agents opens Application Node Governance.

- 1. On the same registry row, select **Agents**.
- 2. Under **Authorized GePNIC Campus Agents**, select only the approved node or nodes, such as TRAIN-01 in this demonstration.
- 3. Choose **Update Node Scope**. Refresh and verify the **AGENT SCOPE** column.
- 4. Confirm with the query owner that the query is approved for the selected source and that the Agent is active and reachable. An Agent missing from this list may not be active; ask infrastructure support to check it.

Two mappings that must not be confused

Mapping	Where to change it	Purpose
API client → Agent(s)	API Client Registry → Agents	Which source nodes this machine key can request.
API user → one Agent	API Subscriptions & Usage → Manage Agent mappings	Subscriber service controls, weekly access and usage attribution.
Human report permissions	Existing reporting governance / User-Agent Matrix	Interactive reporting access; separate from machine access.

Checkpoint

Expected result: the intended Agent appears on the client row. A generated SDK uses the selected client's assigned Agent list. Regenerate and retest the kit after changing its intended scope; editing Java source does not grant additional server permissions.

Set network and data boundaries

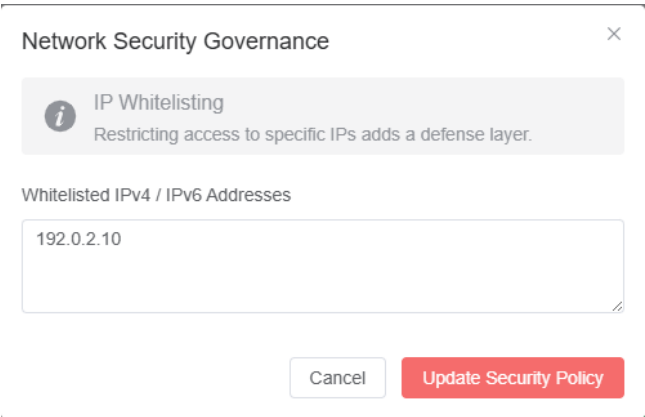


Figure 6. Network Security Governance.

- 1. Select **IP Whitelist**. Enter the approved calling IPs separated by commas, review them, and select **Update Security Policy**. Reopen the dialog to confirm the stored list.
- 2. Review the client's **STATUS** and **ENCRYPTION** columns. Keep the approved active/encrypted configuration. A toggle is an operational change, not a harmless display preference.
- 3. If the data-sharing approval limits the recipient to a particular organization or value, open **Segregation**. For the relevant function, enable the rule, enter the approved JSON field and required value, then select **Save Policies**.
- 4. Confirm the result with an approved test response. DOORS applies segregation on the server before signing/encryption; a successful HTTP response alone is not proof of the correct data scope.

Checkpoint

Use the field name actually returned by the approved query, not a guessed database column. If unsure about the filter, have the query owner confirm it before enabling sharing. Do not remove restrictions merely to resolve a failed call.

Expected result

The client has a recorded source-IP policy, approved query and Agent mappings, intended service status and any required data segregation. The recipient and the Data Manager agree on what a correct sample response should contain.

Choose the query protocol

Protocol versions

TrainingPartnerAPI - Query access and migration

DataManager controls

2 version policies

Retirement dates use your local time.

Migration safety

Legacy versions remain replay-vulnerable while enabled. Save policy changes explicitly; retired versions keep their guide but cannot launch SDK or test actions.

Authorized query	Version / status	Enabled	Deprecated	Retire at	Last accepted	Actions
DailyTenderSummary	V2 Signed freshness Enabled				Not used yet	Save Guide SDK Test
DailyTenderSummary	V1 Legacy Disabled				Not used yet	Save Guide SDK Test

Close

Figure 7. Each query has its own version-policy rows.

- 1. Select **Guide / SDK / Test, Client API** or **Protocol versions** on the client row.
- 2. Find the intended query. For new integrations use its approved **V2 / Signed freshness** row. Check that it is available, enabled and not retired.
- 3. If an approved policy change is needed, set the row's options and select that row's **Save**. Closing the window does not save changes.
- 4. Select **Guide** to read the version-specific contract. Select **SDK** on this same row to choose UAT or Production. **Test** opens the scoped browser test console.

Checkpoint

V1 is legacy and lacks request freshness protection. Do not enable it to work around a V2 failure. A retired/disabled row retains its guide, but its SDK and test actions cannot launch.

What V2 does for you

The SDK signs a request with the client ID, query, a new request ID and current time. Requests expire after five minutes, with 30 seconds of future clock tolerance. Keep workstation clocks synchronized. A repeated ID is rejected; after an uncertain timeout, investigate the original call before retrying.

Choose UAT or Production

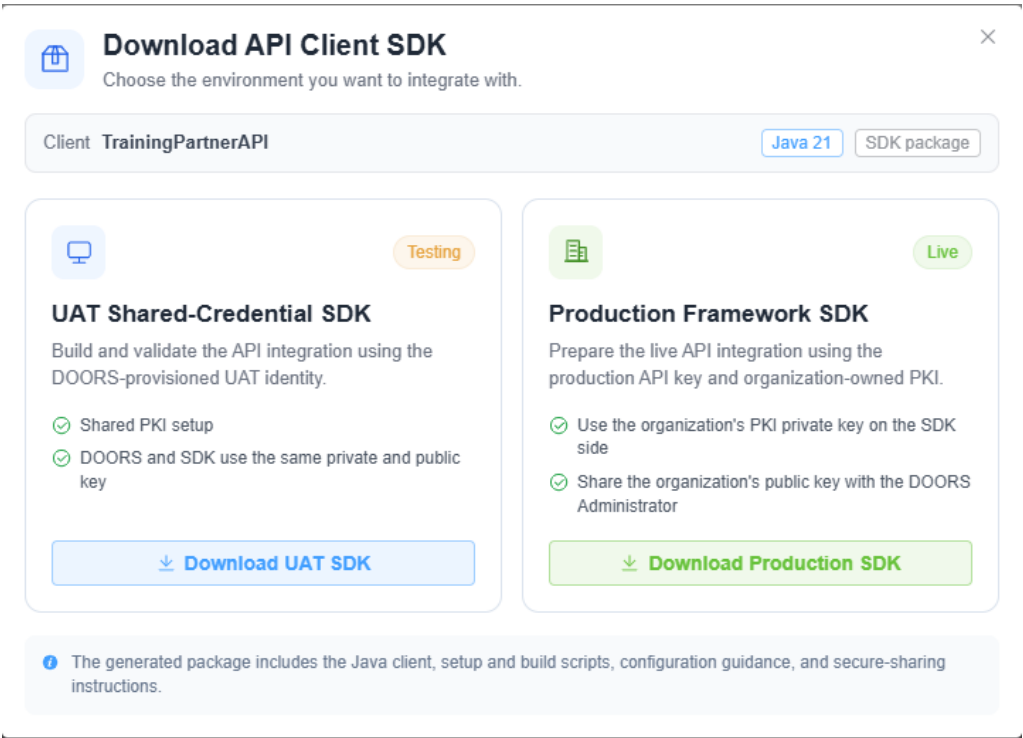


Figure 8. The profile selector opens after SDK on a query/version row.

UAT Shared-Credential SDK	Production Framework SDK
For controlled acceptance testing.	For approved live integration.
Build custodian provisions a private test identity; the UAT kit carries it. DOORS registers the matching public certificate.	Recipient supplies and retains its own private PKCS12 identity; only its public certificate goes to DOORS.
Choose Download UAT SDK.	Choose Download Production SDK.
Contains the selected client API key.	Also contains the selected client API key; it is not a public download.

Checkpoint

One SDK download is scoped to one selected query and protocol version. Neither profile button creates separate environment credentials. Verify the selected client and configured gateway first; provision distinct approved UAT and Production credentials through your onboarding process.

Check the five downloaded files

Open the browser's Downloads list. Allow multiple downloads for your approved portal if prompted, then confirm that all five files arrived. Keep each generation in its own protected folder. Do not mix query versions or clients.

File	UAT name	Production name
Java source	DOORS_SDK_UAT_<Client>_V2.java	DOORS_SDK_PROD_<Client>_V2.java
One-time setup	setup-doors-sdk-uat.ps1	setup-doors-sdk-prod.ps1
Release builder	build-doors-sdk-kit-uat.ps1	build-doors-sdk-kit-prod.ps1
Internal instructions	NIC-DOORS-SDK-BUILD-README.txt	NIC-DOORS-SDK-PRODUCTION-BUILD-README.txt
Draft email	doors-uat-mail.txt	doors-production-mail.txt

How to use these files

Internal README: read this first. It guides the authorized build custodian through setup and release. **Java source:** contains the generated query, Agent list and parameter examples. **Setup:** writes local credentials and compiles the source. **Builder:** creates the shareable release. **Email:** a draft to review, not an automatically sent message.

For TrainingPartnerAPI using V2, the UAT class is **DOORS_SDK_UAT_TrainingPartnerAPI_V2**. Client-name punctuation is removed in the generated class name. Use the exact name in your downloaded Java filename; do not copy this example for another application.

Checkpoint

The first browser download is five separate files, not the final ZIP. For different queries under the same client/version, generated filenames can be identical. Keep separate folders and release records to avoid overwriting or mixing them.

Prepare the build workstation

- 1. Have the approved technical custodian open the protected folder containing the five files. Read the internal README and confirm the client, profile, query, Agent list and gateway URL in the generated source.
- 2. Open PowerShell in that folder: in File Explorer, open the folder, click the address bar, type **powershell** and press Enter. This sets the working directory to your download folder.
- 3. Check the Java tools using the commands below. A JRE alone is insufficient for compilation. If missing or the version is wrong, have support install/configure an approved JDK 21 or later; reopen PowerShell afterward.
- 4. Use a dedicated approved build context for this client/environment. The scripts write to **\$HOME\doors**, which is shared by builds under that Windows account. A different download folder alone does not isolate the credentials.

```
java -version
javac -version
keytool -help
```

Location	Meaning
\$HOME	The Windows account's home folder. PowerShell fills in its actual path.
\$HOME\doors\security	API key and identity/configuration used during setup and testing.
\$HOME\doors\bin	Compiled Java classes.
\$HOME\doors\releases	Default release ZIP, external README, checksum and finalized email.
\$PWD	The current folder. Recipient commands use it to locate the extracted kit.

Checkpoint

Setup writes doors.key and UAT setup can reuse an existing doors-identity.p12. Do not run another client's setup in the same build account midway through a release. Ask the custodian to isolate and verify the build context before proceeding.

Run UAT setup

Use the **UAT** five-file download. Run the setup command from that download folder, following your approved workstation policy:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\setup-doors-sdk-uat.ps1
```

The generated command applies its execution policy only to that PowerShell process. If organizational policy blocks scripts, ask the workstation administrator; do not weaken machine-wide policy.

What successful setup produces

File/folder under \$HOME\doors	Purpose
security\doors.key	Selected client's UAT API key; sensitive.
security\doors-identity.p12	Persistent private UAT test identity; sensitive.
security\doors-wallet.properties	Wallet password/alias configuration; sensitive.
security\client_public.pem	Public certificate to register on this client in DOORS.
src\org\gepnic\doors\masterapi	Generated Java source in its package structure.
bin\org\gepnic\doors\masterapi	Compiled Java classes.

- 1. Wait for the successful compilation message. If any step reports an error, stop and resolve it before building the ZIP.
- 2. Locate **\$HOME\doors\security\client_public.pem**. This is the file used in the next step.
- 3. Keep the private UAT identity within the approved test team. This controlled shared test identity must not become the Production identity.

Checkpoint

Expected result: Java is compiled and client_public.pem exists. The API call is not ready until its matching public certificate is registered against the same API client.

Register the public certificate

Asymmetric Public Key / Certificate Registry

!

Cryptographic Content Audit

Verify external node identities securely using X.509 parameter extractions before committing updates.

OPTION A: Out-of-Band File Provisioning

 Drop your .pem / .crt file here or [browse files](#)

OR MANUALLY PROVISION BELOW

OPTION B: Manual Copy & Paste Plain-Text Base64 Stream

Paste raw Base64 character text block string or —BEGIN CERTIFICATE— data lines straight from your source environment folder...

Cancel

Clear Key Data

Analyze Content

Confirm & Save Key

Figure 9. Analyze the public certificate before saving.

1. Return to **API Client Registry**. Confirm the exact client name/ID and select **Public Key**.
2. Browse/drop **client_public.pem** using Option A, or paste its complete public certificate text using Option B. Do not upload the private .p12 or wallet file.
3. Select **Analyze Content**. Compare organization, common name, issuer, key algorithm and SHA-256 fingerprint with the approved identity details supplied by the custodian/recipient.
4. Only after that check, select **Confirm & Save Key**. Record the client ID and fingerprint. A mismatch must be corrected with the identity owner before testing.

Checkpoint

For UAT use the setup-generated public certificate. For Production use the recipient-owned public certificate. Replacing a client's registered certificate can break an existing integration, so confirm the intended environment and change approval.

Test UAT and build the release

A. Make one approved test call

After certificate registration, use the exact run command from your setup output/generated source and supply the query's required values. Example only:

```
java -cp "$HOME\doors\bin" `
  org.gepnic.doors.masterapi.DOORS_SDK_UAT_TrainingPartnerAPI_V2 `
  'p_summary_date=2026-09-01'
```

A backtick at the end of a line continues the PowerShell command; do not add spaces after it. You can also enter the command on one line. Replace the class name and parameter values for your own download.

Confirm successful authentication, response decryption/signature verification and the expected report data for each authorized Agent. Check that no per-Agent failure is present. Empty data can be a valid result; the query owner should confirm it.

B. Build only after the test is accepted

```
powershell.exe -NoProfile -ExecutionPolicy Bypass `
  -File .\build-doors-sdk-kit-uat.ps1 `
  -UatValidity 'APPROVED-UAT-VALIDITY' `
  -SupportReference 'APPROVED-SUPPORT-REFERENCE'
```

Replace both uppercase placeholders with approved values. These values label the release/email; the builder does not configure server-side expiry or access windows. The default output folder is **\$HOME\doors\releases**.

Output	Check
<ClassName>-<timestamp>.zip	Correct class/client/profile; contains source, classes and UAT credentials.
readme-enduser-uat.txt	Recipient instructions, outside and inside the ZIP.
<full-zip-name>.sha256	SHA-256 fingerprint of this ZIP.
doors-uat-mail.txt	Finalized draft outside the ZIP; still requires human review.

Checkpoint

Do not share a failed build or console output containing sensitive report data. Record the test date/time, query, Agent, sanitized result and approver before release.

Verify the kit and recipient README

1. In the releases folder, match the ZIP filename with its **.sha256** sidecar and the finalized email. Open **readme-enduser-uat.txt** before extracting.
2. Run the checksum command below with the real ZIP filename. Compare the displayed Hash against the sidecar and the email; all must match (letter case does not matter).
3. Extract the complete ZIP into a new protected folder only after verification. Keep the folder structure intact.
4. Open PowerShell in the extracted root, where the recipient README and the **doors** folder sit side by side. Run the README's command with the actual class and approved parameters.

```
Get-FileHash -LiteralPath ".\YOUR-EXACT-RELEASE.zip" -Algorithm SHA256
```

Example recipient UAT command

```
java "-Duser.home=$PWD" -cp ".\doors\bin" `
  org.gepnic.doors.masterapi.DOORS_SDK_UAT_TrainingPartnerAPI_V2 `
  'p_summary_date=2026-09-01'
```

-Duser.home=\$PWD makes the client read credentials from this extracted kit's **doors\security**. Running from the wrong folder is a common reason for missing-key or missing-wallet errors.

Recipient file map

```
extracted-release\
  readme-enduser-uat.txt
  doors\
    bin\...          compiled Java
    src\...          source and integration example
    security\
      doors.key       API key
      doors-identity.p12 UAT private test identity
      doors-wallet.properties
      client_public.pem
```

Checkpoint

If a checksum differs or a file is missing, do not run the kit. Reconfirm the release with the Data Manager through the approved channel. The checksum must come from a trusted release record/email, not only an unverified download location.

Send the UAT handoff email

Open the **doors-uat-mail.txt** in the **releases** folder, not the original unfinished download. The builder fills the ZIP name, checksum, validity and support reference; you must still replace recipient/organization and transfer placeholders. DOORS does not send this email for you.

Subject: DOORS UAT SDK - [application] - [support reference]

Dear [recipient team],

Your approved UAT kit is ready for [application]. It covers [query and protocol] on [Agent scope].

Package: [exact ZIP filename]

SHA-256: [complete hash from the release record]

UAT validity: [approved validity]

Support reference: [reference]

Secure transfer: [approved protected link/channel]

Please retrieve these three files: the ZIP, readme-enduser-uat.txt and the ZIP's .sha256 sidecar. Read the README and verify the hash against both this email and the sidecar before extraction. Stop and contact us if any value differs.

Use JDK 21 or later. Extract the complete kit, open PowerShell at the extracted root and follow the README with your approved parameter values.

The kit contains a UAT API key and private test identity. Keep it restricted to the authorized UAT team; do not forward it or use it in Production. Do not return keys, wallet files, passwords or sensitive report output by email.

Please acknowledge receipt of all three files, name your test contact and send the sanitized acceptance result through [approved support channel].

Regards,

[Data Manager name and approved support contact]

Checkpoint

This is a reusable review template, not a sent message. Transfer the credential-bearing ZIP through the approved protected channel. Send any transfer password/OTP separately. Confirm the support address in the generated draft before using it.

Build the Production Framework SDK

- 1. Obtain Production approval and the Production client/key, query, Agent scope and gateway details. Return to the intended client's **Protocol versions → V2 → SDK** and choose **Download Production SDK**.
- 2. Keep its five files separate from UAT. Read **NIC-DOORS-SDK-PRODUCTION-BUILD-README.txt**. Confirm all mappings, IP restrictions and service controls. Arrange a separate controlled build context.
- 3. Run Production setup, then build using the commands below. Setup writes the selected API key and compiles the source; it does not create the recipient's private identity.
- 4. Verify the ZIP, external README, checksum and finalized email. Check the ZIP contents before secure release.

```
powershell.exe -NoProfile -ExecutionPolicy Bypass `
-File .\setup-doors-sdk-prod.ps1

powershell.exe -NoProfile -ExecutionPolicy Bypass `
-File .\build-doors-sdk-kit-prod.ps1 `
-ProductionValidity 'APPROVED-PRODUCTION-VALIDITY' `
-SupportReference 'APPROVED-SUPPORT-REFERENCE'
```

Inside Production ZIP	Must not be in Production ZIP
Java source and compiled classes	Recipient private doors-identity.p12
doors/security/doors.key (sensitive API key)	doors-wallet.properties / recipient password
readme-enduser-prod.txt	client_public.pem, internal build guide, email draft

Beside the ZIP, the builder writes **readme-enduser-prod.txt**, **<full-zip-name>.sha256** and **doors-production-mail.txt**. Share the first three release files through the approved channel and use the email text for the reviewed handoff.

Checkpoint

The Production kit still contains a secret API key. It must not be publicly hosted. The build can finish before recipient certificate registration; successful compilation is not proof of a successful Production API call.

Recipient installs Production identity

1. The recipient reads **readme-enduser-prod.txt**, verifies the ZIP hash against both sidecar and email, and extracts the complete kit.
2. Its PKI custodian places the organization-owned private identity at **.\doors\security\doors-identity.p12** and creates **.\doors\security\doors-wallet.properties** with the correct alias and password. These files stay with the recipient.
3. The recipient exports only the public certificate as **client_public.pem**. The export command below prompts for the keystore password, avoiding a password in command history.
4. The Data Manager registers this public certificate against the Production client using **Public Key → Analyze Content → Confirm & Save Key** (page 15). Compare its fingerprint through the approved onboarding channel.
5. Before the first live call, the recipient's technical custodian supplies the approved TLS trust store and confirms gateway hostname/certificate trust. Run the generated Production command from the extracted root and validate the agreed sample response.

```
# Recipient-only protected wallet file
DOORS_WALLET_PASS=YOUR_ACTUAL_P12_PASSWORD
DOORS_WALLET_ALIAS=YOUR_ACTUAL_CERTIFICATE_ALIAS

# Export the public certificate; replace YOUR_ALIAS
keytool -exportcert -rfc `
  -keystore ".\doors\security\doors-identity.p12" `
  -alias "YOUR_ALIAS" `
  -file ".\doors\security\client_public.pem"
```

Checkpoint

Current implementation detail: Production expects doors-truststore.p12 under doors/security (or approved DOORS_TRUSTSTORE_PATH / DOORS_TRUSTSTORE_PASS configuration). The generated setup/build does not provision this trust store. Have the PKI team supply/configure it before testing; the signing P12 and TLS trust store serve different purposes.

Do not email the private P12, wallet file, API key or password to DOORS. For a NIC-side technical test, use a separately controlled approved test identity/client. Replacing the live registered public key for a test can disrupt the recipient.

Send the Production handoff email

Use the generated **doors-production-mail.txt** from the release folder. Its demo download URLs are template values; replace them with the three verified approved protected locations. Confirm the application name as well as filename, hash, validity and reference.

Subject: DOORS Production Framework SDK - [application] - [reference]

Dear [recipient team],

The approved Production Framework SDK for [application] is ready.

Package: [exact ZIP filename]

SHA-256: [complete verified hash]

Validity/approval: [approved details]

Support reference: [reference]

Retrieve the release through these approved protected locations:

1. SDK ZIP: [protected ZIP link]
2. readme-enduser-prod.txt: [README link]
3. ZIP .sha256 sidecar: [checksum link]

Read the external README and verify the ZIP checksum against both this email and its sidecar before extraction. The ZIP contains your API key and is restricted to authorized staff.

Install your organization-owned PKCS12 identity and protected wallet configuration locally. Arrange the approved TLS trust-store configuration with your PKI team. Send only client_public.pem through the approved onboarding channel for DOORS registration; retain the private P12 and password internally.

Run the SDK only after DOORS confirms certificate registration and query/Agent authorization. Do not send keys, wallet files, passwords or sensitive response data by email.

Please acknowledge all three release files, confirm the technical contact and agree the first-call validation time with us.

Regards,

[Data Manager name and approved support contact]

Checkpoint

DOORS generates email text; it does not deliver email or upload the kit to the sample URLs. Record the recipient, approver, checksum, transfer method/time and acceptance result in your release register.

Test through the browser when needed

The Java SDK is the primary end-to-end onboarding route in this manual. The browser console is useful for an approved diagnostic call with technical support.

- 1. From the intended client, open **Protocol versions** and choose **Test** on the approved query/V2 row. This preserves the selected query/version context. The registry also provides **Test API**.
- 2. Allow the approved test window if the browser blocks it. Launch from the registry rather than bookmarking a direct Swagger URL; DOORS creates a scoped launch session.
- 3. Use the console's local PKCS12 file and password controls with the matching approved identity. Enter the actual parameter values and build the encrypted request using the console controls.
- 4. Review query, Agent, parameters and target environment. Execute only the intended test request. Opening the console alone does not run the query.
- 5. Inspect the HTTP result and verified/decrypted response. Record a sanitized result and close the console when finished. Keep raw response data within approved handling rules.

Acceptance checks for either SDK or browser test

Check	Pass condition
Identity	The intended client key and registered certificate authenticate.
Query and Agent	Only the approved query and source scope are used.
Parameters	Required names and real approved test values are supplied.
Data	Query owner confirms expected fields, rows and segregation.
Protection	Response decryption and signature verification succeed.
Operation	No per-Agent error; usage and access status are consistent.

Checkpoint

A portal login is not a substitute for machine API authorization. Do not change the whitelist, disable encryption or enable legacy V1 simply to get a green test result.

Link the API user and subscription

The screenshot shows a web interface titled "Agent mapping & service · audit-subscriber". It contains two dropdown menus: the first is labeled "API user" and has "audit-subscriber" selected; the second is labeled "Map API user to GePNIC Agent" and has "GePNIC test instance (TEST-01)" selected. Below these is a summary box titled "audit-subscriber → TEST-01" which states "1 API clients covered by this mapping" and includes a link "GePNIC test integration". A paragraph of text explains that exactly one agent is licensed for this API user and that suspension applies to every API key mapped to that agent. Below this is a toggle switch for "DOORS API service" which is currently set to "Enabled". A final paragraph states that subscription periods and payment amounts are managed separately. At the bottom right are "Cancel" and "Save Agent mapping & service" buttons.

Figure 10. Manage Agent mappings opens a visible dialog. This separate fixture uses audit-subscriber and TEST-01.

1. Open **Governance Hub → API Subscriptions & Usage** and choose **Manage Agent mappings**. Search/select the intended API user in the dialog.
2. Set the approved GePNIC Agent and manual service state, then choose **Save Agent mapping & service**. One API user maps to one Agent. The API client's source mapping remains in the Registry.
3. Open **Subscriptions & Agent mapping**. Use **Add subscription period** for a renewal; enter approved dates and GePNIC/DOORS amounts in INR and select **Add period**.
4. Use **Edit period** to record payments or correct that period, then **Save period changes**. Do not replace an earlier period just to record a renewal; periods must not overlap.

Checkpoint

Subscription dates and balances are advisory: they do not automatically suspend service. The manual service switch does affect machine requests for every key mapped to that Agent. Shared Agent/client mappings can affect more than one recipient, so review the scope before changing service state.

Set access hours and monitor usage

Weekly access - audit-subscriber

All API keys mapped to GePNIC test instance follow this schedule. Times are always India time (IST), regardless of your browser's time zone.

Unrestricted - 24 x 7

Scheduled hours

Quick fill

Mon-Fri - 09:00-18:00

Every day - all day

Clear all slots

Sunday

22:00

to

Monday

02:00

Add slot - Sunday

Remove slot

Monday

No slot starts here. A slot from the previous day may continue into this day.

Add slot - Monday

Tuesday

No slot starts here. A slot from the previous day may continue into this day.

Add slot - Tuesday

Wednesday

No slot starts here. A slot from the previous day may continue into this day.

Add slot - Wednesday

Thursday

No slot starts here. A slot from the previous day may continue into this day.

Add slot - Thursday

Friday

No slot starts here. A slot from the previous day may continue into this day.

Add slot - Friday

Saturday

No slot starts here. A slot from the previous day may continue into this day.

Add slot - Saturday

The end is the next occurrence of the chosen day and time. Sunday 22:00 → Monday 02:00 allows overnight access; Saturday → Sunday wraps into the next week. Start inclusive, end exclusive. Overlapping slots are rejected.

Cancel

Save weekly access

Figure 11. Weekly access editor with synthetic schedule data.

- 1. Open the **Weekly access** tab and choose **Edit weekly access** for the intended user. Select **Unrestricted - 24 x 7** or **Scheduled hours**.
- 2. For scheduled access add the approved slots with start day/time and end day/time, then **Save weekly access**. Weekly times use India time. A scheduled week with no slots closes all hours.
- 3. Review **Existing date-specific client restrictions** as well. A weekly schedule does not remove them. Date-calendar entry uses the browser timezone shown in its dialog; the dashboard displays saved times in India time.
- 4. On **Usage & scorecards**, select the reporting dates and **Apply dates**. Expand the API user to inspect individual clients, requests, data sent, records and failures. Check **Alerts & notices** for subscriber messages.

Checkpoint

All applicable controls must allow the request: service enabled, permitted weekly hours and any date-specific calendar. Start is inclusive; end is exclusive. An empty date-specific calendar is unrestricted, unlike an empty scheduled week. Notices are portal messages, not email delivery.

Usage dates are inclusive in India time. Data sent counts application response-body bytes, including errors, before proxy compression; it is not network billing or proof that the recipient consumed the data. Earlier activity is unavailable before tracking began.

DOORS | API DATA SHARING | 18 SEP 2026

24 / 27

When something does not work

Symptom	First action
Cannot sign in / wrong role	Check the admin portal, current verification code and authenticator. Same-host tabs share the session. Ask support about account/role issues.
Query missing or disabled in selection	Confirm Approved status and API-compatible name without spaces. Resolve through query governance; do not create a duplicate workaround.
Agent missing / SDK unavailable	Confirm an active Agent mapping, authorized query and enabled, available, unretired protocol row.
Only one download arrived	Check browser multiple-download permissions and Downloads. Keep all five files from the same generation.
java / javac not recognized	Confirm JDK 21+ and JAVA_HOME/Path with support; reopen PowerShell after configuration.
Missing key / wallet / class	Check current folder, complete extraction, exact class name and -Duser.home=\$PWD. Verify setup/compile succeeded.
TLS certificate or trust-store error	Check approved gateway hostname, certificate chain and trust-store path with PKI support. Do not bypass verification.
Authentication / signature failure	Check the selected client, current API key, registered public-certificate fingerprint and matching private identity. Share no secrets.
403 service/window problem	Check manual suspension, every applicable weekly schedule and client calendar. Expired subscription dates alone do not suspend.
400 expiry / 409 duplicate in V2	Check system time and original request result. Do not blindly generate a new request ID after an uncertain timeout.
Hash mismatch	Do not extract/run. Reconfirm ZIP, sidecar and trusted email against the same release.
HTTP success but wrong/empty data	Ask the query owner to check parameter values, source Agent and segregation; check per-Agent failures too.

Checkpoint

Support bundle: timestamp with timezone, client ID/name, query/version, Agent, Java version, HTTP status/problem code, trace ID if present and sanitized error. Exclude API keys, private files, passwords, raw sensitive responses and authenticator details.

Final handoff and acceptance checklist

Confirm before release	Record / tick
Correct recipient, application, environment and approval reference	_____
Client ID, approved query/version and Agent scope checked	_____
Source IP policy and any segregation rule checked	_____
API user mapping, service state and applicable hours checked	_____
Correct public certificate and fingerprint registered	_____
Gateway and TLS trust confirmed; Production trust store prepared	_____
Approved test call verified by query/data owner	_____
ZIP, correct recipient README and .sha256 verified	_____
Production ZIP excludes recipient private identity and wallet	_____
Email placeholders, support contact and secure links reviewed	_____
Recipient acknowledged all three files and checksum	_____
First recipient call and acceptance recorded	_____

Keep a release record

Application/client ID; environment; query and protocol; Agent scope; build date and custodian; verifier; ZIP filename and SHA-256; certificate fingerprint; approved validity; recipient/contact; support reference; secure transfer method/time; test outcome; acceptance date; and any outstanding issue. Keep secrets outside this record.

After onboarding

Monitor client usage and failures. Coordinate API key/certificate renewal or scope changes with the recipient, regenerate and retest affected kits, and keep previous release records. If credentials are exposed, follow the organization's incident and revocation process immediately.

Checkpoint

Completion means the recipient has made and accepted an authorized, verified call. A successful download, ZIP build or email send alone is not the end of onboarding.

References and current guide notes

Files this manual explains

Audience	Read these files
UAT build custodian	NIC-DOORS-SDK-BUILD-README.txt; setup-doors-sdk-uat.ps1; build-doors-sdk-kit-uat.ps1; generated Java source.
UAT recipient	readme-enduser-uat.txt, supplied beside and inside the release ZIP.
Production build custodian	NIC-DOORS-SDK-PRODUCTION-BUILD-README.txt; setup-doors-sdk-prod.ps1; build-doors-sdk-kit-prod.ps1; generated Java source.
Production recipient	readme-enduser-prod.txt, supplied beside and inside the release ZIP.
Data Manager handoff	Finalized doors-uat-mail.txt or doors-production-mail.txt in the release folder.

Check these environment-specific details

Gateway and credentials: profile selection does not provision a new environment. The generated source uses the configured machine gateway and selected client key. Confirm both before release.

Production trust: the runtime expects an approved PKCS12 trust store/configuration; the generator does not build it. Have technical support supply this prerequisite (page 20).

README examples: current UAT text contains example dates, a p_summary_date example and named test organizations. Use the actual query inputs and intended recipient. Verify the checksum before extraction even if an earlier example paragraph says to unzip first.

Email templates: current Production text contains demoetenders.tn.nic.in example download URLs. They are not proof of upload or authorization to publish. Replace all links and recipient/application placeholders; confirm the support address.

Implementation references

This manual was checked against src/views/auth/LoginView.vue; src/views/governance/ApiClientManagement.vue (registry, protocol, SDK, README and email generators); QuerySubmission.vue; QueryLibrary.vue; ApiSubscriptionDashboard.vue; docs/API_SUBSCRIPTIONS.md; docs/API_USER_PORTAL.md; backend ApiClientController.java; and static doors-swagger.html. Screenshots show the current local UI with synthetic API responses; they are illustrations, not evidence of live onboarding.

The editable companion is docs/DOORS_API_Data_Sharing_Visual_User_Manual.md. Update this manual when the UI labels, protocol, generated package contents or operating process changes.